Coding - Introduction

Emergency numbers

India

- 108, 100

UK

- 999, 112

They are not 8 digit or 10 digit numbers.
Why are they 3 digit numbers?

Is that all?

- The emergency telephone number may differ from country to country.
- It is typically a three-digit number so that it can be easily remembered and dialed quickly.
- Some countries have a different emergency number for each of the different emergency services; these often differ only by the last digit.

Can you attach any other reason?

Can you answer this question?

- Say you are assigning telephone numbers to 100 persons.
- You have 100 numbers with you but they are not of same length
  - Few numbers are 2 digit numbers – remaining are 10 digits number
- In the 100 persons, PM, and President are also there, & some common people are also there.
- **<u>How do you assign the numbers? Who 'll get the 2 digit number?</u>**

How many bits do we need?

- M symbols denoted s_1; s_2; : : : ; s_M
- Simple English text: 26 letters of the alphabet + one space i.e. M = 27
- Images and video: 256 gray levels i.e. M = 256
- How many bits do we need to represent?
- $\text{Log}_2(M)$ – rounded to next immediate higher integer
- If M=8 then we need 3 bits
- If M=27 then we need 5 bits

Assumption

- All the symbols represented by equal bits
- For example,
  - a as 5 bit length code (say 00000)
  - z also as 5 bit length code (say 11010)
- Fixed length code

Another type …

- Variable length coding
- Each symbol represented by different code lengths
- For example,
  - a as 5 bit length code (say 00000)
  - z as 8 bit length code (say 11010000)

Why do we need variable length code?

<u>Uniform distribution</u>

- All the symbols are equi-probable go for fixed length code

<u>Non-uniform distribution</u>

- Variable length coding
- Reduces average code length

Example

	Probability
S_1	0.25
S_2	0.25
S_3	0.25
S_4	0.25

	Code
S_1	00
S_2	01
S_3	01
S_4	11

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 2; Length of S_2 = 2; Length of S_3 = 2; and Length of S_4 = 2
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4

= 0.25*2 + 0.25*2 + 0.25*2 + 0.25*2 = 2 bits

Example

	Probability
S_1	0.5
S_2	0.25
S_3	0.125
S_4	0.125

	Code
S_1	00
S_2	01
S_3	01
S_4	11

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 2; Length of S_2 = 2; Length of S_3 = 2; and Length of S_4 = 2
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4

= 0.5*2 + 0.25*2 + 0.125*2 + 0.125*2 = 2 bits

Example

	Probability
S_1	0.5
S_2	0.25
S_3	0.125
S_4	0.125

	Code
S_1	0
S_2	10
S_3	110
S_4	111

- 4 symbols: Log(M) = 2 bits
- Length of S_1 = 1; Length of S_2 = 2; Length of S_3 = 3; and Length of S_4 = 3
- Average = $P(S_1)$*Length of S_2 + $P(S_2)$*Length of S_2 + $P(S_3)$*Length of S_3 + $P(S_4)$*Length of S_4

= 0.5*1 + 0.25*2 + 0.125*3 + 0.125*3 = 1.75 bits

Another example

- Given 4 symbols
- Need to be transmitted
- 3 designs are proposed
- Which one to choose?

Average code length

Symbols	P_k	Code 1	Code 2	Code 3
S_0	0.5	0	0	0
S_1	0.25	1	1 0	0 1
S_2	0.125	0 0	1 1 0	0 1 1
S_3	0.125	1 1	1 1 1	0 1 1 1

Find the average for code 1, code 2 and code 3
Weighting factor = their probability values

Average bits of code 1= 0.5*1+0.25*1+0.125*2+0.125*2=1.25
Average bits of code 2= 0.5*1+0.25*2+0.125*3+0.125*3=1.75
Average bits of code 3= 0.5*1+0.25*2+0.125*3+0.125*4=2.375

Efficient coding of a random message

- Example

  You would like to set up your own telephone system that connects you to your three best friends

  The question is how to design efficient binary phone numbers

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

In this design, Alice and Bob have the same phone number

The system obviously will not be able to connect properly

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

This is much better, i.e. the code will actually work.

However, the phone numbers are quite long and therefore the design is rather inefficient

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

- Much shorter and, at the same time, we have made sure that we do not use the same codeword twice
- System will not work
- Code **<u>is not uniquely decodable</u>**
- if you dial 10 this could mean “Carol” or also “Bob, Alice.”
- Or, in other words, the telephone system will never connect you to Carol, because once you dial 1, it will immediately connect you to Bob

Uniquely decodable

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

Quite efficient code that is functional
But we note something: when calling Alice, why do we have to dial two zeros?
After the first zero it is already clear to whom we would like to be connected

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

This is still uniquely decodable and obviously more efficient than (iv)

•Is it the most efficient code?
•No! Since Bob is called most often, he should be assigned the shortest codeword

Friend	Probability	Phone number					
Alice	1/4	0011	001101	0	00	0	10
Bob	1/2	0011	001110	1	11	11	0
Carol	1/4	1100	110000	10	10	10	11
		(i)	(ii)	(iii)	(iv)	(v)	(vi)

This is the optimal code

Note one interesting property: even though the numbers do not all have the same length, once you finish dialing any of the three numbers, the system immediately knows that you have finished dialing

achievable average code-word length.

- In practice, optimal codes like Huffman codes or Shannon-Fano codes can approach this limit, though they may not always achieve it exactly due to the discrete nature of code-word lengths.

Efficiency (η)

- The efficiency η of a coding scheme is defined as the ratio of the source entropy to the average code-word length:

$$\eta = \frac{H(S)}{\bar{L}}$$

- Since $\bar{L} \geq H(S)$, the efficiency η is always less than or equal to 1 ($\eta \leq 1$).
- An efficiency of $\eta = 1$ indicates a perfectly efficient code where the average code-word length equals the entropy, which is the theoretical maximum efficiency.

Implications

1. **Lower Bound on Compression**: The theorem establishes that the entropy $H(S)$ is the minimum number of bits per symbol needed to encode the source losslessly. Any attempt to compress below this limit would result in information loss.
2. **Practical Coding**: Real-world coding schemes can get close to $H(S)$ but are limited by the fact that code-word lengths must be integers. This is why the average length $\bar{L}$ is often slightly greater than $H(S)$.

Shannon's first theorem : Source-coding theorem

- Given a dms of entropy H(S), the average code-word length $\overline{L}$ for any source coding is

$$\overline{L} \geq H(S)$$

i.e.) $$L_{\min} = H(S) \qquad \& \qquad \eta = \frac{H(S)}{\overline{L}}$$

Shannon's Source-Coding Theorem, also known as the first theorem in information theory, is a fundamental result established by Claude Shannon. It provides a theoretical limit on the efficiency of lossless data compression for a given source of information. Let's break it down step by step based on the image you provided:

Key Concepts

1. **Discrete Memoryless Source (DMS) and Entropy (H(S))**:
   - A discrete memoryless source is a probabilistic model where each symbol emitted is independent of the previous symbols, and the probability distribution of the symbols is fixed.
   - The entropy $H(S)$ of a source S measures the average uncertainty or information content per symbol, expressed in bits (if using base-2 logarithm). It is defined as:

$$H(S) = -\sum_i p(s_i) \log_2 p(s_i)$$

   where $p(s_i)$ is the probability of symbol s_i, and the sum is over all possible symbols in the source alphabet.

2. **Average Code-Word Length ($\bar{L}$)**:
   - This represents the average number of bits required to encode a symbol from the source when using a particular source coding scheme (e.g., Huffman coding or arithmetic coding).
   - The theorem addresses the efficiency of this encoding process.

The Theorem

The theorem states:

- The average code-word length $\bar{L}$ for any source coding scheme must be at least as large as the entropy $H(S)$ of the source. Mathematically:

$$\bar{L} \geq H(S)$$

- This means that no lossless compression scheme can reduce the average code-word length below the entropy, as entropy represents the fundamental limit of compressibility.

Special Case: Optimal Coding

- The equality $\bar{L} = H(S)$ holds when the coding scheme is optimal (i.e., it achieves the minimum possible average code-word length). This is denoted as $L_{\min} = H(S)$, where $L_{\min}$ is the minimum achievable average code-word length.

code-word lengths must be integers. This is why the average length $\bar{L}$ is often slightly greater than $H(S)$.

3. **Redundancy**: The difference $\bar{L} - H(S)$ represents the redundancy in the code, which is the extra bits used beyond the theoretical minimum.

Example

Suppose a source S has two symbols, A and B, with probabilities $p(A) = 0.75$ and $p(B) = 0.25$. The entropy is:

$$H(S) = -[0.75 \log_2(0.75) + 0.25 \log_2(0.25)] \approx 0.811 \text{ bits}$$

- An optimal code might assign A a 1-bit code (e.g., 0) and B a 2-bit code (e.g., 1), leading to an average length $\bar{L} = 0.75 \cdot 1 + 0.25 \cdot 2 = 1.25$ bits.
- Here, $\bar{L} > H(S)$, but as the block size increases (e.g., encoding sequences of symbols), $\bar{L}$ can approach $H(S)$ more closely.

Conclusion

Shannon's Source-Coding Theorem provides a foundational limit for data compression, guiding the design of efficient coding schemes. It tells us that the entropy $H(S)$ is the ultimate benchmark for lossless compression, and any practical system must strive to minimize the gap between $\bar{L}$ and $H(S)$ to maximize efficiency. This theorem is a cornerstone of information theory and has widespread applications in data compression, telecommunications, and beyond.